Amazon Athena

Explore how Amazon Athena allows you to run serverless SQL queries directly on data stored in Amazon S3 without managing infrastructure. Understand its support for multiple data formats, integration with AWS Glue for metadata management, and best practices for optimizing performance and security. Gain the skills to build scalable, cost-efficient analytics workflows using Athena within AWS ecosystems.

Amazon Athena is a serverless, pay-per-query service that enables developers to analyze structured and semi-structured data in Amazon S3 using **ANSI SQL**. It eliminates the need to manage infrastructure or provision resources, allowing teams to focus entirely on querying data and building insights. It is especially useful for quickly analyzing large-scale datasets without a complex setup.

Athena is well-suited for developers building analytics, reporting, or serverless data processing capabilities into their applications. It's fast, scalable querying combined with minimal operational overhead makes it a practical choice for modern data-driven workloads. It supports a wide range of common data formats such as CSV, JSON, ORC, Parquet, and Avro, allowing developers to query data directly in its raw form without extensive preprocessing. This versatility is particularly useful in S3-based data lake architectures.

Ask

Beyond SQL querying, Athena also provides *Apache Spark support*, enabling users to run interactive analytics in environments like Jupyter Notebooks. This expands its appeal to data analysts and engineers who need advanced processing capabilities without managing infrastructure.

Integration with S3

Amazon Athena can be conveniently integrated with several Amazon services. However, the trademark use case is with S3 buckets. As Athena is serverless and compatible with multiple formats, it is ideal for performing ad-hoc SQL queries on data stored in S3. It is commonly used for quick data exploration, troubleshooting (e.g., analyzing web logs), or any scenario where we need to analyze S3 data using interactive SQL queries without managing servers.

Athena uses a **Hive-compatible Data Definition Language (HiveQL DDL)** to define metadata about datasets, allowing it to efficiently interpret and query data stored in S3.

Analyzing data in S3 with Amazon Athena

A common use case for Athena is analyzing AWS service logs stored in S3, such as AWS CloudTrail logs. CloudTrail logs capture API activity and events within an AWS account, and they are typically stored in S3 in a highly partitioned fol structure, often by date. This partitioned format helps optimize querying by

Ask

allowing Athena to scan only relevant data based on time-based filters. For instance, our CloudTrail logs might be organized as follows:

main.sh Shell

```
s3://<bucket_name>/cloudtrail/year=2025/month=06/day=15/
```

CloudTrail logs stored in S3

To start querying CloudTrail data stored in S3 with Athena, we first need to create a database. In Athena, a database acts as a logical container for one or more tables. It helps organize our queries and metadata. We can create a database using the following SQL command:

> **Tip:** The SQL commands can be executed using Athena Query Editor.

main.sql MySQL

```
CREATE DATABASE <DatabaseName>
```

Ask

Create a table

Once the database is created, the next step is to define a table that tells Athena how to interpret the data stored in S3. As Athena doesn't automatically infer the structure of S3 objects, we need to explicitly describe the schema, data format, and partitioning layout.

In this case, we'll define an external table that maps to the S3 path containing CloudTrail logs and specifies how Athena should parse the JSON data. The following `CREATE EXTERNAL TABLE` statement provides this definition:

main.sql MySQL

```
CREATE EXTERNAL TABLE cloudtrail_logs (
  eventVersion STRING,
  userIdentity STRUCT<arn:STRING>,
  eventTime STRING,
  eventName STRING,
  responseElements STRUCT<LoginStatus:STRING>
)
PARTITIONED BY (year STRING, month STRING, day STRING)
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'
STORED AS TEXTFILE
LOCATION 's3://<bucket_name>/cloudtrail/';
```

Ask

Create Athena table

- **Lines 1–7:** We define an external table and specifying the fields to extract from each log record.
- **Lines 8–9:** We declare the partitions by `(year,month,day)` to align with the folder structure and configuring the input as JSON using a standard SerDe to parse the data.

After the table is created, we need to sync the partitions. This makes Athena aware of existing folders like `year=2025/month=06`. We can sync the table by using the following query:

main.sql MySQL

```
MSCK REPAIR TABLE cloudtrail_logs;
```

Load partition

`MSCK REPAIR TABLE` compares the partitions in the table metadata and the partitions in S3. If new partitions are present in the S3 location that we specify when the table is created, it adds those partitions to the metadata and to the Athena table.

Ask

Once the table is set up, we can query the table using Athena. Here's an example SQL query to find failed login attempts over a specific period:

main.sql MySQL

```sql
SELECT userIdentity.arn, eventTime, eventName
FROM cloudtrail_logs
WHERE eventName = 'ConsoleLogin'
  AND responseElements.LoginStatus = 'Failure'
  AND year = '2025' AND month = '06';
```

Get login attempts

Here we are applying filters on partition keys (`year`, `month`) to ensure that Athena scans only relevant files and only required columns are queried, avoiding `SELECT *` and minimizing cost. Partition filtering reduces the amount of data scanned, improving performance and lowering cost.

Integration with AWS Glue Data Catalog

Amazon Athena is a powerful tool for querying data directly in S3, but manually defining tables for each dataset can quickly become cumbersome, especially with numerous data sources and teams. This is where AWS Glue comes in, streamlining the process and acting as a crucial link between our raw S3 data and Athena's query engine.

AWS Glue automates schema discovery and metadata management, significantly simplifying data operations. With Glue, we can:

Ask

- Automatically detect schemas, file format, and partitions of our data in S3.

- Maintain a consistent metadata catalog that can be reused across various AWS services, such as Athena, Redshift Spectrum, and EMR, ensuring data consistency and accessibility.
- Automating schema detection and metadata management reduces maintenance overhead. We eliminate the need to hardcode schemas, drastically reducing manual effort and potential errors.

By leveraging AWS Glue with Athena, we can efficiently query our S3 data without the complexities of manual table definitions, allowing us to focus on data analysis rather than infrastructure management.

Athena and Glue Data Catalog

There are two main ways to populate the catalog:

- **Glue Crawlers to populate the catalog:** A Glue Crawler scans the S3 data, detects the format, schema, and partition structure, and then creates a table in the Data Catalog. Here's how we define one via AWS CLI:

main.sh Shell

```
aws glue create-crawler \
  --name <crawler_name> \
  --role <Role_name> \
  --database-name <database_name> \
  --targets S3Targets=["Path=s3://<bucket_name>/logs/"]
```

Ask

Create a crawler using AWS CLI

Once the crawler runs, the resulting table is available in Athena under the specified Glue database.

- **Manually define the table:** While Glue helps with automatic schema generation, partition discovery, and table registration, sometimes that's not exactly what we need. When we're working with datasets that have nested structures, irregular formatting, or require performance-critical tuning, we would prefer to define the schema manually using `CREATE EXTERNAL TABLE` in Athena. Generally, manual definition is preferred when:
  - The dataset is complex and requires precise control over SerDe settings.
  - The schema is not reliably inferable (e.g., nested or inconsistent JSON).
  - We need to align schema logic across multiple services or audit processes.

> **Tip:** The Glue Crawlers are ideal for dynamic data or automated ingestion pipelines, and manual definitions are ideal when we need tight control over schema details or SerDe parameters.

Integration with other services

Amazon Athena can also integrate with other AWS resources:

- **Amazon QuickSight:** Generate data visualizations from Athena query for easy data exploration.

Ask

- **Business intelligence (BI) tools and SQL clients:** Integrate Athena with existing BI tools or SQL clients using JDBC/ODBC drivers for broader data analysis capabilities.

Performance optimization strategies

Optimizing query performance in Amazon Athena is essential for minimizing cost, improving responsiveness, and staying within service limits. As Athena charges based on the amount of data scanned, efficient querying directly impacts both budget and performance.

Here are key strategies to write high-performance Athena queries:

- **Use columnar and compressed formats:** Store data in columnar formats like *Apache Parquet* or *ORC*, which allow Athena to read only the columns needed for a query. These formats are highly efficient for analytical queries and dramatically reduce scan size. Pairing them with compression algorithms like *Snappy* or *GZIP* further reduces the amount of data transferred from S3 during queries, improving both speed and cost-efficiency.

> Avoid formats like JSON or CSV for large datasets unless absolutely necessary.

Ask

- **Partition data:** Partitioning is the process of organizing data in Amazon S3 into separate folders based on column values (e.g., `year=2024/month=07/`).

This allows Athena to scan only the relevant subsets of data rather than the entire dataset. It is recommended to partition by columns that are commonly used in `WHERE` clauses, such as `year` or `region`. However, over-partitioning, using highly granular columns like `user_id`, can lead to increased metadata and slower query performance.

- **Use partition pruning in queries:** Partition pruning is a query optimization technique where Athena reads only the partitions that match the filter conditions in the query. Without partition filtering, Athena may scan all partitions, leading to higher cost and slower performance. To enable pruning, queries must include partition key conditions in the `WHERE` clause. The example below shows how specifying `year` and `month` allows Athena to scan only the corresponding folders in S3:

main.sql MySQL

```sql
SELECT user_id, event_time
FROM logs
WHERE year = '2025' AND month = '06';
```

Query Athena by partition pruning

- **Avoid `SELECT *`:** Using `SELECT *` causes Athena to read all columns, even if we're only using a few, increasing the scanned data size.
- **Enable partition projection for predictable structures:** When the partition structure is predictable, like time-based, we can enable partition projection.

Ask

This avoids querying Glue for every partition and speeds up planning for large datasets.

- **Understand the query patterns using `EXPLAIN`:** Before running a complex query, use `EXPLAIN` to understand how Athena will execute it. This helps us identify whether filters, joins, or scans will behave as expected. Here's how we can use `EXPLAIN` to preview a query:

main.sql MySQL

```
EXPLAIN SELECT * FROM logs WHERE year = '2025';
```

Preview query

- **Use CTAS (Create Table As Select) for reusable results:** For large, frequently used queries, we can materialize the result into a compressed, optimized columnar format using CTAS. This lets us avoid re-executing the same logic repeatedly and allows downstream systems to access the optimized output. The code below creates a reusable table for summary metrics:

main.sql MySQL Ask

```
CREATE TABLE summary_logs
WITH (format = 'PARQUET', external_location = 's3://<bucket_name>/sum
```

```
SELECT user_id, COUNT(*) as event_count
FROM logs
WHERE year = '2025'
GROUP BY user_id;
```

CTA for reuseable results

Once created, future queries can run against `summary_logs`, significantly reducing query time and cost.

Securing queries and outputs

Security in Amazon Athena is a multi-layered responsibility. Since Athena reads data directly from Amazon S3 and uses AWS Glue for metadata, developers must secure access to data, query results, and metadata.

Below are key strategies and configurations to help us protect Athena workloads.

- **Control access with IAM:** We can use fine-grained IAM policies to control access to Athena APIs, specific S3 buckets or prefixes, Glue tables and columns.
- **Encryption:** Athena writes all query results to a specified S3 location. If this location isn't secured, sensitive outputs may be exposed. All Athena

Ask

communication with S3 uses TLS by default. To ensure the data is secure at rest, we can encrypt S3 with the encryption options provided for S3.

- **Fine-grained access with AWS Lake Formation:** AWS Lake Formation provides centralized, column-level access controls on top of the Glue Data Catalog. Lake Formation integrates with Athena transparently without modifying queries. With Lake Formation we can control which users or roles can access specific tables or columns.

By combining these practices, we can build secure and compliant Athena-based analytics workflows.

Serverless workflow integration

Amazon Athena fits naturally into serverless and event-driven architectures. Developers can invoke Athena queries programmatically using the AWS SDK, integrate them into Lambda functions, or orchestrate workflows using Step Functions.

- **Lambda and Athena:** Lambda can process streaming or batch data written to S3. For example, when application logs are uploaded to an S3 bucket, a Lambda function can be triggered via an S3 `PUT` event. This Lambda then runs a predefined Athena query to filter for anomalies, failed login attempts, or error events.

Lambda integration with Athena

Ask

- **Step Functions:** Athena query execution is asynchronous. Using Step Functions allows us to manage this workflow cleanly. A state machine can

first invoke an Athena query, wait for the result using the `GetQueryExecution`, and then pass the result to a Lambda function for transformation or archiving. We can also define retries, timeouts, and error handlers for robust control.

Step Functions integration with Athena

- **EventBridge and Athena:** We can schedule Athena queries using EventBridge rules. This pattern is useful for periodic reporting or data freshness checks.

Troubleshooting and debugging queries

Athena query issues typically stem from missing permissions, mismatched schemas, unoptimized queries, or invalid data formats. Common errors include `Access Denied` from insufficient IAM roles or `HIVE_PARTITION_SCHEMA_MISMATCH` from inconsistent partition structures.

To resolve these, ensure the IAM policies include access to the relevant S3 paths and Glue tables and use `MSCK REPAIR TABLE` to sync partitions when needed.

We can also enable CloudWatch Logs to capture execution errors and query diagnostics. During development, it is better to test queries with small datasets using `LIMIT` to validate logic efficiently.

Ask

Conclusion

Amazon Athena offers a powerful, serverless solution for querying large-scale datasets directly from Amazon S3 using standard SQL. With support for various

Ask